

Proyecto Elegido:

Audiometría



Integrantes: Benjamin Elias Saragusti, Agustin Ortiz, Agustin Lerman y Samir Rivera

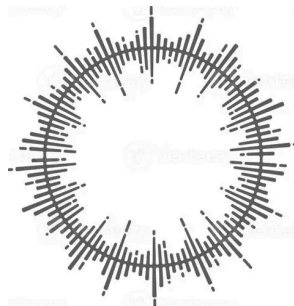
Curso: 4 2 Computación

Profesor: Gonzalo Consorti

Año:2025

Objetivo:

El objetivo del dispositivo creado es el de a través de la reproducción de músicas por medio de un reproductor mp3 llamado DF player lograr que se generen patrones de luces y colores con la utilización de un NEOPIXEL para demostrar los cambios de volumen en el dispositivo.



Investigación:

En lo que a mi respecta investigue cómo se conectaba el DF PLAYER al arduino, de los puertos que tiene utilizamos:

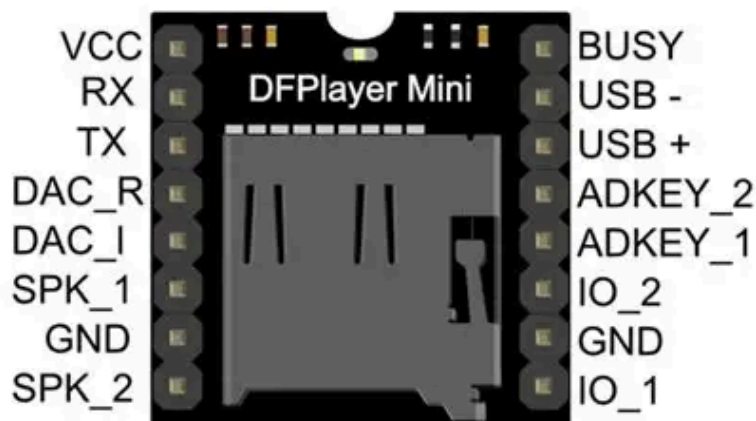
SPK1: Esta es la salida para el altavoz, por este puerto sale la mitad del sonido para parlantes, pero esta al estar amplificada no es recomendable para conectar al arduino porque al tener mucha señal podría llegar a quemar este dispositivo.

SPK2: La salida complementaria a SPK1, aplica lo mismo que el anterior.

GND: Esta es la salida a tierra, la que se conecta al negativo del arduino para cerrar el circuito y descargar energía.

VCC: Se conecta en este caso a 5V, pero al nuestro proyecto contar con varios dispositivos que requieren de energía se usa una placa de energía por separado que da 5V pero con mejor estabilidad.

RX y TX: Son los puertos que se conectan al arduino para enviar y recibir señales seriales de este, que junto con una librería permiten darle órdenes al DF PLAYER mediante la utilización de este equipo.



DFROBOT PLAYER: Este es el nombre de la librería que usamos, la cual nos permite darle órdenes como:

```
#include "DFRobotDFPlayerMini.h"
```

```
Funciones principales
myDFPlayer.next(); //Reproduce el siguiente archivo mp3
myDFPlayer.previous(); // Reproduce el anterior archivo mp3
myDFPlayer.play(1); //Reproduce el primer archivo mp3
myDFPlayer.loop(1); //Reproduce infinitamente el primer archivo mp3
myDFPlayer.pause(); //pause
myDFPlayer.start(); //reproduce desde la pausa
```

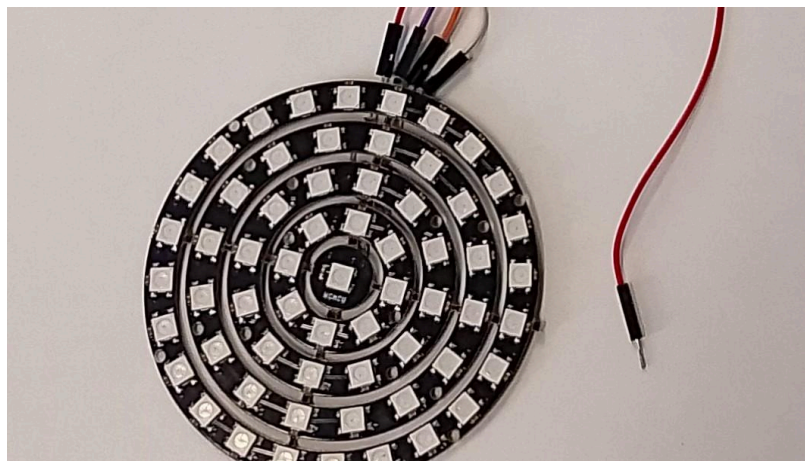
Sin esta librería el Arduino no podría haber interpretado las señales del df player.

Ahora toca la explicación del NEOPIXEL:

VCC: La entrada de energía de 5V para este círculo de leds.

GND: Entrada de negativo

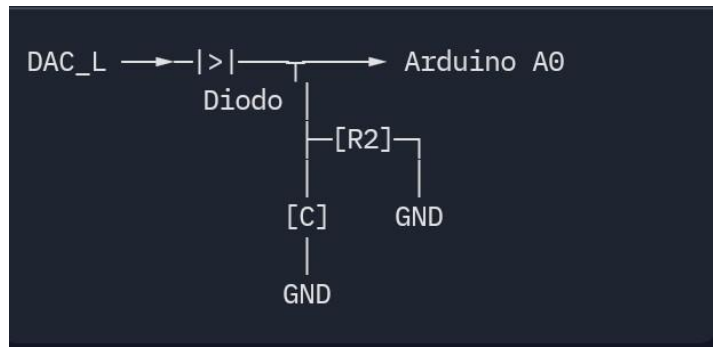
IN: Este se conecta a uno de los puertos de entrada digital/salida analógica para darle órdenes de cómo se van a prender los leds y con qué color se hará.



ADAFRUIT NEOPIXEL: Esta es la librería que requerimos para poder interpretar las órdenes del NEOPIXEL.

```
#include <Adafruit_NeoPixel.h>
```

Aparte del circuito de lectura analógica de las señales de audio de DF PLAYER



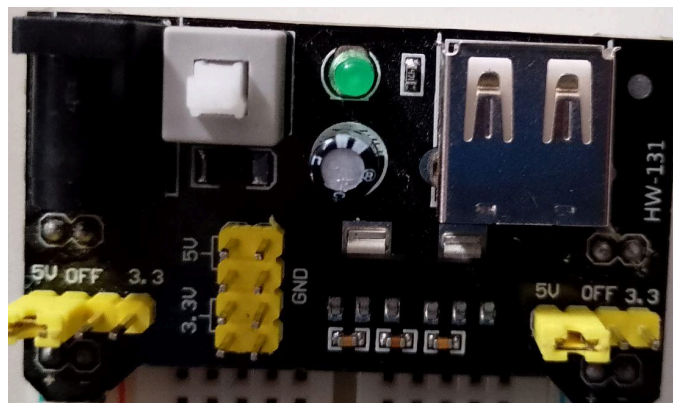
R2:Resistencia

C:Condensador

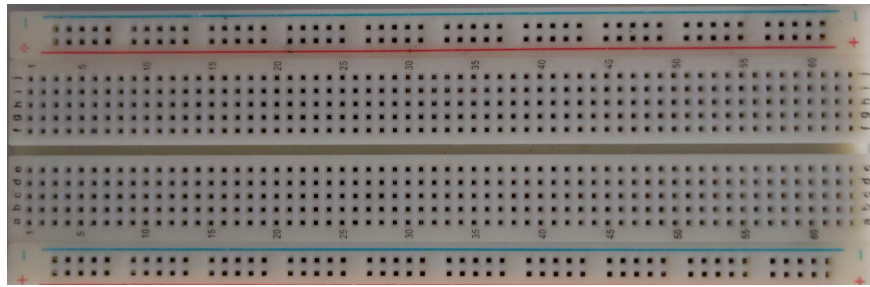
Este esquema es el cual envía las señales desde el df player al arduino para calcular la cantidad de audio detectado por este reproductor mp3. El diodo se usa para asegurar que el arduino no se queme, la resistencia también colabora con esto haciendo que la descarga al gnd no sea tan grande por lo que protege al arduino. Y el condensador sirve para estabilizar las señales y lograr unos campos más precisos que no se podrían de otra forma sin este

Componentes utilizados:

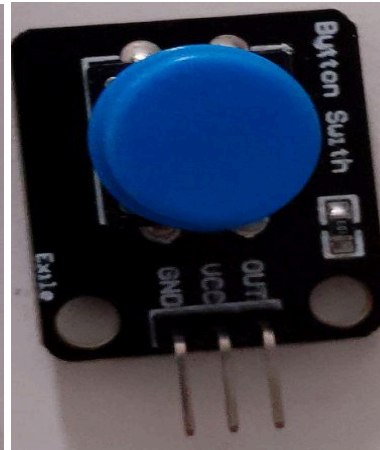
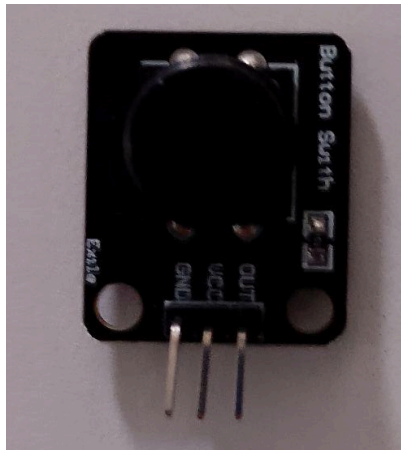
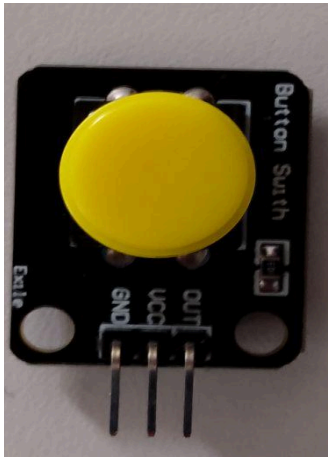
Fuente alimentación



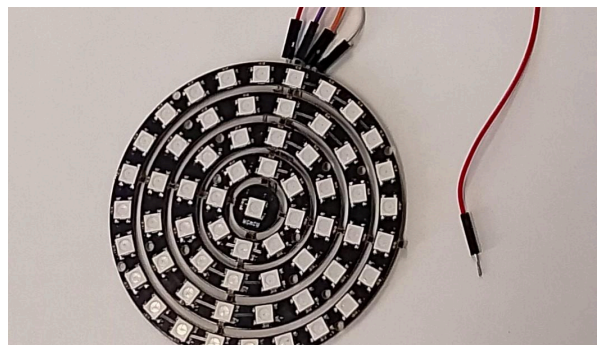
Protoboard:



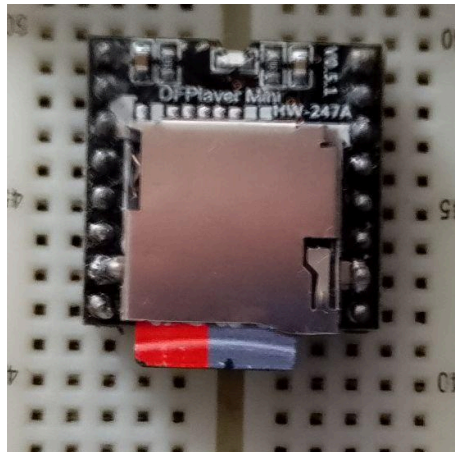
Botones(3)



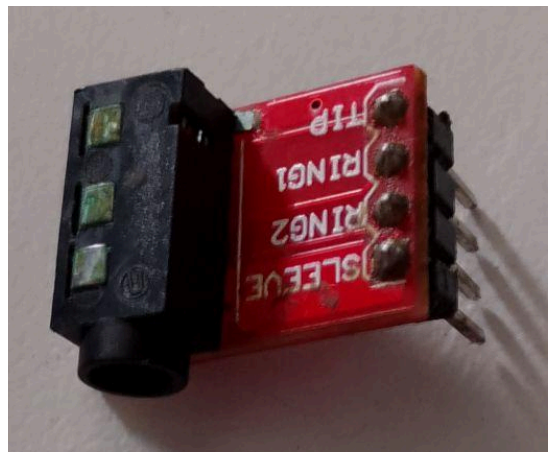
Neopixel (61 led)



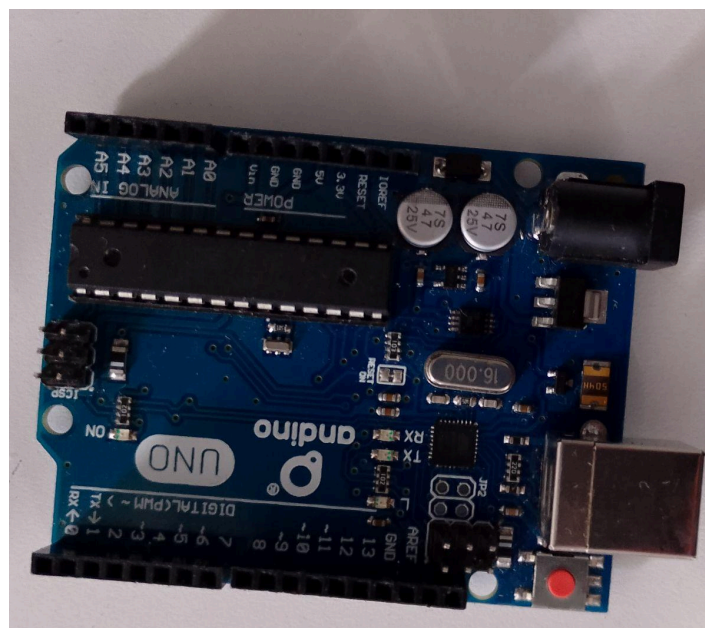
DF PLAYER



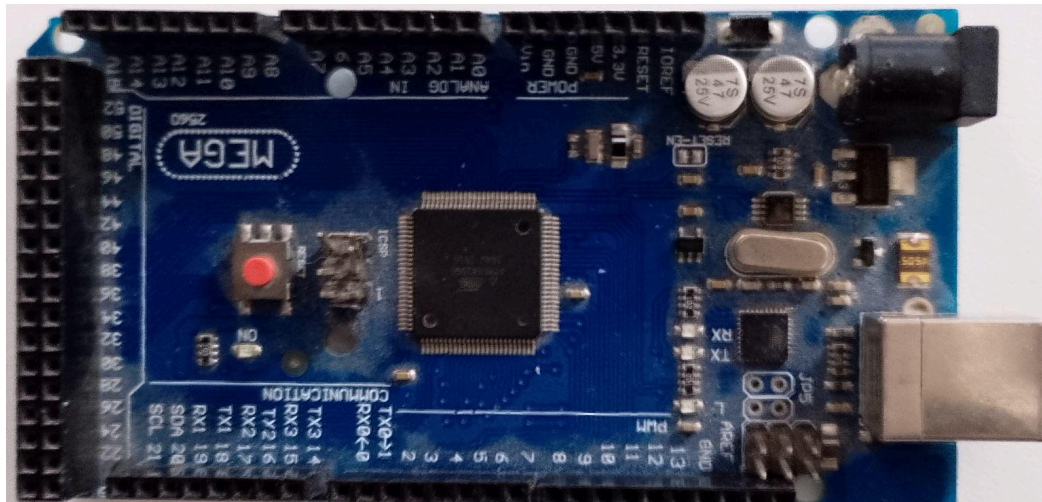
Adaptador a AUX



ARDUINO UNO

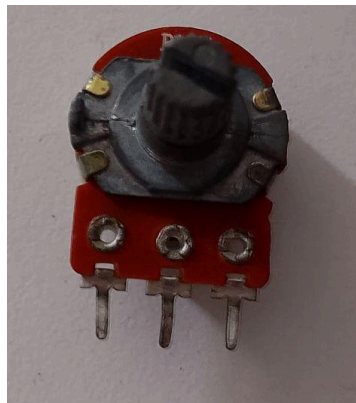


ARDUINO MEGA



Componentes que fueron descartados:

Potenciómetro (Se descartó para la función de volumen pero se usó para probar el neopixel)



Condensador pf 10



Resistencia 1K OHM



DIODO



Pruebas realizadas/Avances con respecto al código:

Lo primero en lo que me enfoque fue en realizar un código para poder cambiar de canciones, con un límite de hasta ocho. Este se ejecuta usando dos funciones, una que te reinicia el valor a 0 en el caso de que haya llegado a 8 y otra que suma de 1 para reproducir. De esta manera cada vez que el botón se detecta como pulsado se da una señal la cual activa este código y hace que cambie de canción.

```

int Mas(int Numero){
  if (Numero==8){
    Numero=0;
  }
  return Numero;
}

int Cambio (int Numero2){
  if(Numero2!=0){
    Numero2=Numero2+1;}else{
    Numero2=1;
  }
  return Numero2;
}

```

El botón está configurado de manera INPUT_PULLUP lo que significa que cuando lo presiones va a cambiar temporalmente de 1 a 0 su valor, estando en 1 cuando está inactivo y 0 al estar activo.

```

#include "Arduino.h"
#include "DFRobotDFPlayerMini.h"
int contador=0;
#define Boton_Derecha 5
void setup()
{
  FPSerial.begin(9600);
  Serial.begin(115200);
  pinMode (Boton_Derecha, INPUT_PULLUP);
}

```

Con respecto al NEOPIXEL realice código que implementa un cambio de cómo se veían los colores de este cada vez que se cambiaba el número indicador de la cantidad de sonido que ejecutaba la música.

```

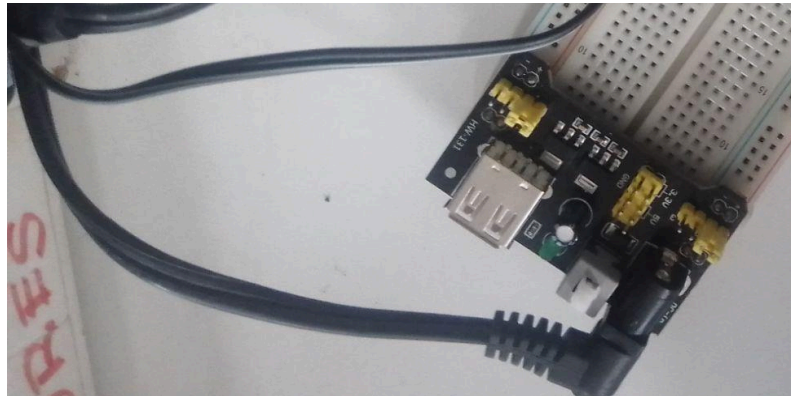
void ceroacienaudio (){
  pixel.fill (pixel.Color(0,0,255), 0,60);
  pixel.show();
  for (int i=0;i<length (ceroa100, int);i=i+1){
    pixel.setPixelColor (ceroa100[i] ,pixel.Color (0,255,0));
  }
  pixel.show ();
}

```

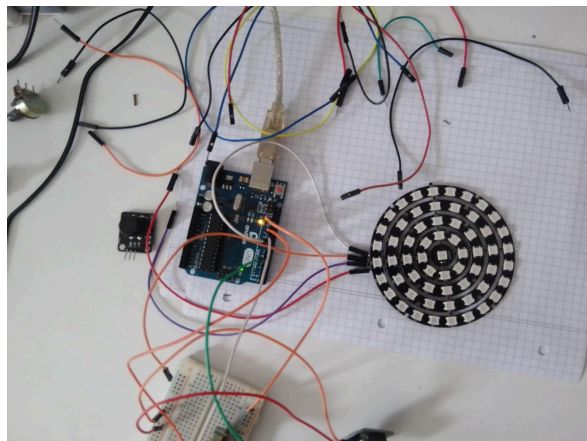
(Una de las configuraciones que yo hice)

En parte de lo técnico y más físico tuvimos que usar una placa de energía porque los 5V que mandaba el arduino no eran suficientes para suplir las necesidades presentadas por

componentes varios, esta se colocaba en el extremo superior y se conectaba al toma corriente por medio de un cable.



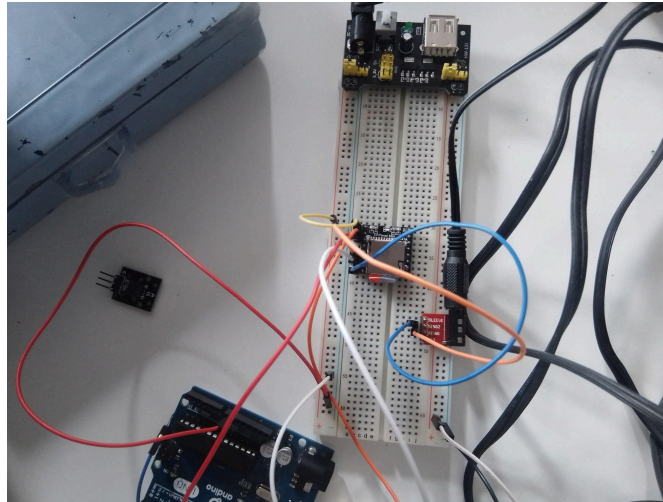
Para probar que los comandos fueran identificados por el arduino y el neo pixel en conjunto utilice un potenciómetro conectado al puerto A1 que simula las señales del 0 al 1023 que debería de enviar el df player, por lo que empezó a demostrar un cambio de estado que no era el esperado pero que se notaba la diferencia entre los cambios, al no contar con el circuito de lectura de df player para ese momento se usó un potenciómetro para simular los valores.



Después una vez con el código del df player ya finalizado probé los tres botones con el arduino y sus tres funciones funcionaron siendo:

Parar por completo la canción, una pausa temporal y otro que pasa a la siguiente canción.

Para escuchar las canciones tuvimos que utilizar un adaptador a aux que se conectan los puertos spk1 del df player al ring 1 del adaptador aux y el puerto spk2 a ring 2, enviando las señales a un parlante mono.



Modelo en 3d:

Aunque directamente no hice el diseño en 3d aporte medidas aproximadas sobre los componentes anotando en una hoja, la cual a día de hoy no cuento con ella porque se la di a mis compañeros. Además de haber dado las medidas de los componentes propuse el diseño el cual se basa en un celular, colocando el neopixel en la posición superior, los dos arduinos en la parte inferior junto con la protoboard y los objetos en ella y los botones también junto al neopixel. De esto tampoco tengo la hoja ya que también se las cede a ellos pero en el croquis que realice se podía apreciar cómo se deben de distribuir los componentes y en qué distancias, ósea en altura junto con los debidos soportes requeridos.

Contratiempos:

Primero fue el tema de que el código base que usamos para entender el funcionamiento del df player tenía los datos de tx y rx al revés, por lo que cuando conectamos los cables para probarlo por primera vez dio error y no se pudo ejecutar.

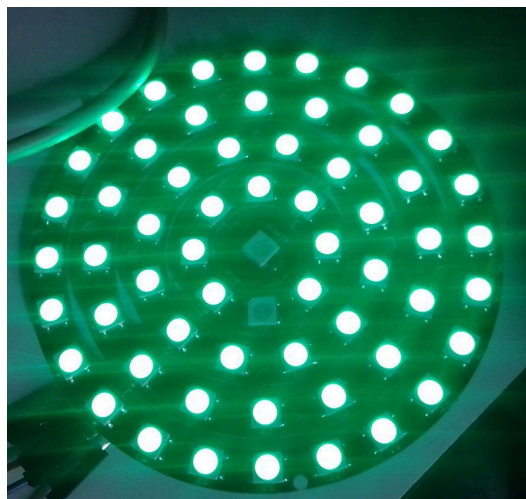
```
| SoftwareSerial mySoftwareSerial(3, 4); // tx, rx
```

Segundo era el tema de que cuando implementamos el sistema para que se aumente o se disminuya el volumen del Arduino pero se tuvo que eliminar debido a que no sonaba en ningún momento.

```
int volumen = analogRead (pot);
int volume = map (pot,0,1023,0,30);
delay (20);
myDFPlayer.volume(volume);
```

Tercero los potenciómetros que use para probar lo que sería la lectura faltaban o estaban rotos, solo uno funciono de verdad pero tuve que estar probando distintos hasta encontrar uno que de verdad funcione.

Cuarto, al juntar ambos códigos con las partes de todos (que me encargue yo) las señales del neopixel y del df player se corrompen haciendo que este último no reproduzca música y que el Neopixel no mostrase los patrones de imagen. Por lo que tuvimos que dividir el código para que se ejecutará de manera individual en cada dispositivo aparte.



Quito: Al intentar hacer la conexión de un arduino a otro usando un circuito de resistencia, diodo y condensador las señales eran aleatorias y con poco rango de distancia por lo que no se podía manejar de manera precisa, por lo que tuve que descartar la idea y usar un generador aleatorio para que mostrara las animaciones.

```
int Lectura=analogRead (Df_lectura);
delay (100);
```

Resultado Final:

Al final este dispositivo quedó con la funcionalidad de Pausar, Pausar temporalmente y cambiar de canción, además de representar el sonido en patrones dentro del neopixel.

Con un modelo en 3D que no pudo ser impreso por la escasez de tiempos pero que se pudo llegar a terminar, tomando de inspiración un celular.

